GroundingEval

Claim-Level Grounding Evaluation for Multimodal Scene Understanding

CityOS Agentic Stack

Winlab Summer 2026

Center for Smart Streetscapes (CS3)

Rutgers University

June 2026

Contents

Read First. Read `PROJECT_OVERVIEW.pdf` before this document. It explains the full CityOS agentic stack, the compiler analogy, and how all teams fit together.

Note on LLMs. The student-built evaluation infrastructure is **entirely deterministic**. The checkers, QA generator, and labels never call an LLM. You do not need OpenAI or Anthropic API keys for the evaluation core.
The research verification layer (claim extraction) does use an LLM to translate TeLLMe's natural-language answers into structured claims. However, once claims are structured, the verification is deterministic Python code that checks each claim against the ground-truth JSON. The principle remains the same as everywhere in the stack: deterministic checkers verify the output of non-deterministic generators.

1 Before You Start

Before writing any code or schemas, you need to understand what the detector produces and what TeLLMe answers look like. The CityOS `eye-in-the-sky` application is the primary detector whose output you will work with.

1.1 Clone and Explore CityOS

```
git clone https://github.com/Center-for-Smart-Streetscapes-CS3/cityos.git
cd cityos
```

Read the directory `apps/eye-in-the-sky/` carefully. Understand how the detector ingests camera frames, runs inference, and produces output JSON. Pay attention to the fields it emits per frame.

1.2 The eye-in-the-sky Output Format

The detector produces one JSON object per processed frame.

```
{
  "input_timestamp_ms": 1718000000123,
  "timestamp_ms": 1718000000456,
  "detected_objects": [
    {
      "box": [120.5, 200.3, 250.1, 400.8],
      "class_id": 0,
      "class_name": "pedestrian",
      "confidence": 0.92,
      "track_id": 7
    },
    {
      "box": [500.0, 180.0, 700.0, 350.0],
      "class_id": 2,
      "class_name": "vehicle",
      "confidence": 0.87,
```

```
      "track_id": 12
    }
  ],
  "metrics": {
    "backend": "tensorrt",
    "inference_ms": 8.3,
    "cuda_memory_allocated_mb": 412
  }
}
```

Field explanations.

`box` Bounding box as `[x1, y1, x2, y2]` pixel coordinates.

`class_name` Human-readable class (e.g., "pedestrian", "vehicle").

`confidence` Detection confidence between 0.0 and 1.0.

`track_id` Persistent identifier from BoT-SORT. Same physical object keeps the same ID across frames.

2 The Problem. Why Grounding Matters

Large language models are fluent but unreliable. When TeLLMe answers a question about a scene, it produces a natural-language sentence that sounds correct regardless of whether it actually is. "A person is sitting at the desk" reads the same whether or not there is a person in the room. The model does not distinguish between things it knows from sensor data and things it is guessing. This is the hallucination problem, and it is the central obstacle to deploying LLM-powered systems in real-world applications where wrong answers have consequences.

The obvious response is to check the answer. But check it *how*? You could ask a second LLM whether the answer seems right. This is called LLM-as-judge, and it is the approach most research papers use. The problem is that the judge has the same failure modes as the generator. If the generator hallucinates "two people are standing near the door," a judge model might agree because the sentence is plausible. You have replaced one unreliable system with two unreliable systems.

The alternative is to check the answer against **structured ground truth** using **deterministic logic**. You define what is true about the scene (one person, sitting, at the desk) as a structured representation. You decompose TeLLMe's answer into individual claims ("a person is sitting," "at the desk," "using a laptop"). You check each claim against the ground truth with simple Python functions. There is no ambiguity, no judgment call, no second LLM. Either the claim matches the ground truth or it does not, and when it does not, you can say exactly which ground-truth fact was violated.

This is what GroundingEval builds.

3 TeLLMe Is the Application Under Test

TeLLMe is the system that looks at multimodal observations from the Smart Room or CARLA simulator and answers questions about what is happening. GroundingEval is the system that checks whether those answers are correct. The relationship is simple: TeLLMe is the application being tested, and GroundingEval is the test harness.

The data flows in one direction.

```
Smart Room / CARLA simulator
        |
        v
sensor observations (images, video,
  object states, metadata)
        |
        v
TeLLMe
        |
        v
natural-language answer
        |
        v
GroundingEval
        |
        v
deterministic correctness check
  against ground truth
```

Concrete example. A camera in the Smart Room observes one person sitting at a table with a backpack on a chair.

Observation. One person sitting at a table. Backpack on a chair.

Question to TeLLMe. "Is anyone sitting in the room?"

TeLLMe answer. "Yes, one person is sitting at the table."

GroundingEval checks:

- Does the ground truth say there is a person? **Yes.**
- Does it say the person is sitting? **Yes.**
- Does it say the person is at or near the table? **Yes.**

Result. All claims SUPPORTED.

GroundingEval does not need to know *how* TeLLMe arrived at its answer. It does not need to understand TeLLMe's internals, its prompts, its model, or its reasoning chain. It only needs two things: TeLLMe's answer in a known format, and a ground-truth description of the scene.

3.1 Who Does What

TeLLMe is the perception and question-answering application. It observes the scene, receives a question, and produces an answer. It is built by Team 4.

GroundingEval is the evaluator and test harness. It checks TeLLMe's answers against structured ground truth. It is built by this team.

Oscar's research layer handles claim extraction (parsing TeLLMe's natural-language answers into structured claims) and explores formal verification approaches for complex claims.

Student infrastructure covers scenarios, ground truth, QA pairs, and deterministic scoring. This is the bulk of the work.

3.2 GroundingEval Does Not Depend on TeLLMe Being Finished

This is critical. TeLLMe is being built in parallel by another team. It may not be stable, complete, or even running when you start building GroundingEval. **That is fine.** GroundingEval should work with *mock TeLLMe outputs* from day one.

A mock TeLLMe output is just a JSON file with hand-written answers to your QA pairs. You write the questions, you write the answers (some correct, some wrong), and you run your evaluation pipeline on them. This lets you build and test your entire pipeline without waiting for anyone.

When TeLLMe is ready, you swap mock answers for real answers. The pipeline does not change. Only the input changes. If your pipeline works on mock answers, it will work on real answers.

4 What GroundingEval Is

GroundingEval is the **test harness for grounded multimodal QA**. TeLLMe observes a scene (through Smart Room sensors or CARLA simulation) and answers questions about it. GroundingEval checks whether those answers are correct by comparing them against structured ground truth.

The central question is whether, when TeLLMe says "A person is sitting at the desk," we can determine that this claim is correct and point to the specific ground-truth facts that support it.

GroundingEval does **not** decide truth. It does not observe the world. It compares a TeLLMe answer against a previously defined ground-truth representation using deterministic logic. If the ground truth is wrong, the evaluation will be wrong. But the evaluation logic itself is inspectable, deterministic, and auditable, unlike an LLM-as-judge.

4.1 Where GroundingEval Fits in the Stack

The CityOS agentic stack has a core design principle: *deterministic checkers verify the output of non-deterministic generators.* TraceFix uses TLC (a deterministic model checker) to verify coordination protocols generated by an LLM. ConcordFS uses deterministic FIFO channel validation to enforce message ordering. GroundingEval extends this principle to the *output* of the system. TraceFix verifies that agents coordinate safely. GroundingEval verifies that what the agents *say about the world* is actually true.

Think of it this way. TraceFix asks: "Is this multi-agent coordination protocol safe?" GroundingEval asks: "Is this answer about the scene correct?" Both questions are answered by deterministic checkers, not by asking another LLM.

Without GroundingEval, TeLLMe has no feedback signal. It generates answers, but nobody checks whether those answers match reality. Without GroundingEval, there is no way to measure progress,

no way to catch regressions, and no way to demonstrate to a user that TeLLMe's answers are trustworthy.

4.2 What You Own

You own four modules that together form a complete evaluation pipeline.

Scenario module. Creates or records a world state. A scenario is a snapshot or sequence of a physical scene or simulated environment. What objects exist, where they are, what they are doing, and what changed over time. This is the "test case" that everything else is built on.

QA module. Turns the world state into test questions. Given a ground-truth state with specific objects and relations, the QA generator produces questions that TeLLMe should be able to answer ("Is there a person in the room?") along with the expected answers ("yes") and the ground-truth facts that justify each answer. This is the "test input."

Answer module. Stores TeLLMe's actual answer. What did TeLLMe say, was it boolean, count, relation, or free text, and can we normalize it into a form the checkers can process. This is the "test output."

Scoring module. Checks the answer. Did TeLLMe's answer match the expected answer, and what kind of mistake was it. This is the "test verdict."

4.3 What You Interface With

You interface with Smart Room (sensor streams, detector outputs), TeLLMe (answers to your prompts), CARLA (simulated scenarios with known ground truth), and Oscar's research verification layer. You do not build or modify CityOS, ConcordFS, or TraceFix.

Smart Room and CARLA provide the raw material: scenes with objects in them. TeLLMe provides the answers you are evaluating. You provide the ground truth, the test questions, and the scoring, and you report back to TeLLMe and Smart Room how well the system performed.

4.4 The Name

This project is called **GroundingEval**. The name reflects what it does: it evaluates whether TeLLMe's answers are *grounded* in observable, verifiable facts about a scene. "Grounding" means tying a claim to specific evidence. "Eval" means scoring it deterministically. The full name in papers and presentations is *GroundingEval: Claim-Level Grounding Evaluation for Multimodal Scene Understanding.*

5 The Broader Picture. Smart Room Evaluation Harness

GroundingEval tests whether TeLLMe's answers are *true*. But truthfulness is not the only thing that matters. A deployed CityOS application also needs to be *useful*, *private*, *fast*, and *compliant* with the correct API tier. These are different questions, and they should be tested separately.

The right framing is that GroundingEval is one component of a broader **Smart Room Evaluation Harness**.

```
Smart Room Evaluation Harness
```

```
|
+-- GroundingEval
|     Is the answer correct?
|
+-- UtilityEval
|     Is the answer useful for the task?
|
+-- PrivacyEval
|     Did the system obey CityOS privacy constraints?
|
+-- PerformanceEval
|     Was it fast enough?
|
+-- PolicyEval
      Did it use the right CityOS API tier and data path?
```

The CityOS paper already provides the template for the extra dimensions. API 1 is evaluated using sniffing exposure and latency overhead. API 2 is evaluated using DP counting accuracy (RMSRE) and sensitivity. API 3 is evaluated using origin-destination estimation utility and self-identification quality. The graphs on pages 11–13 of the CityOS paper are exactly this kind of privacy/utility evaluation.

5.1 What Students Build This Summer

You do **not** build the full evaluation harness this summer. You build GroundingEval (the correctness component) and the shared scenario infrastructure that all future evaluation tracks will use. The shared infrastructure includes the scenario generator, ground-truth state format, QA generator, TeLLMe answer collector, deterministic scorer, metrics logger, and report generator.

The key design decision is that every scenario should include **extra fields beyond just "is the answer correct."** This summer you only check whether TeLLMe's answer matches the ground truth. But later, someone will want to check whether the answer was fast enough, whether the system respected privacy rules, or whether it used the right CityOS API. If you put placeholder fields for those things in your scenario JSON now (even if you leave them empty), the next team does not have to redesign the file format. You are future-proofing the schema.

For the 12-week project, the team should fully implement:

- Grounding correctness (the core)
- Basic utility metrics (task success, answer coverage, false alarm rate)
- Basic privacy/policy compliance checks from logs (which API tier was used, whether raw data left the node)

Oscar can own the deeper CityOS-style privacy/utility curves as a research direction.

5.2 What GroundingEval Directly Tests

GroundingEval tests the following error types. Each one is a specific, named way that TeLLMe's answer can be wrong.

Correctness. The answer matches ground truth.

Hallucination. TeLLMe asserts something that does not exist in the scene.

Omission. TeLLMe fails to mention something that does exist.

Wrong count. TeLLMe says 2 people, ground truth has 1.

Wrong relation. TeLLMe says the backpack is on the table, it is on the chair.

Wrong state. TeLLMe says the person is standing, the person is sitting.

Wrong temporal event. TeLLMe says the person entered at 10:03, they entered at 10:01.

Unsupported answer. TeLLMe makes a claim that cannot be checked against available ground truth.

Ambiguous answer. TeLLMe's answer is vague enough that it could be either correct or incorrect.

These are the error categories that your evaluation report should contain. Every evaluated claim should be tagged with one of these.

6 The Claim-Level Pipeline

The naive approach to evaluation is string matching: compare TeLLMe's answer to an expected answer. This fails immediately. TeLLMe might say "Yes, there is one person sitting down" and the expected answer might be "1." These mean the same thing, but string matching says they are different. You could add fuzzy matching, but then you are writing heuristics that are hard to maintain and easy to game.

The claim-level approach solves this by decomposing answers into structured, checkable units. Instead of comparing two strings, you extract individual **claims** from TeLLMe's answer and check each one against the ground truth independently. A claim is a single factual assertion: "there is a person," "the person is sitting," "the person is at the desk." Each claim has a clear ground-truth counterpart, and checking it is a function call that returns true or false.

```
Scenario (real or simulated)
  -> Ground-truth state JSON (objects, relations, attributes)
  -> Generated QA pairs (from templates)
  -> TeLLMe answer (natural language)
  -> Claim extraction (structured claims)
  -> Deterministic evaluator
  -> SUPPORTED / CONTRADICTED / NOT_EVALUABLE
  -> Metrics and report
```

For simulator scenarios (CARLA), ground truth comes from the simulator state. The simulator knows exactly which actors exist and where they are, so ground truth is free and automatic. For physical Smart Room scenarios, ground truth comes from human-entered scenario scripts or manual labels. Someone watches the video or sets up the scene and writes down what is true. This is more expensive, but it is the only way to get ground truth for the real world.

7 Data Model

The data model is the most important design decision in the entire project. It determines what kinds of claims you can check, how precise your evaluation can be, and how easy it is for other teams to produce data you can use. A bad data model means months of painful workarounds. A good one means everything clicks into place.

The key insight is that ground truth should be **structured as objects and relations**, not as free text. If ground truth is "a person is sitting at the desk," you cannot programmatically check whether TeLLMe's answer matches it without doing natural-language understanding, which is exactly the problem you are trying to evaluate. But if ground truth is a JSON object with `person_1`, type `person`, state `sitting`, location `chair_1`, then checking is a dictionary lookup.

7.1 Ground-Truth State

Each scenario has a ground-truth state describing the world. This is the "answer key" that all evaluation is measured against. It is a structured representation, not a sentence.

```
{
  "scene_id": "smartroom_001",
  "timestamp_range": ["2026-06-15T10:00:00Z",
                      "2026-06-15T10:05:00Z"],
  "objects": [
    {"id": "person_1", "type": "person",
     "state": "sitting", "location": "chair_1"},
    {"id": "chair_1", "type": "chair"},
    {"id": "laptop_1", "type": "laptop",
     "location": "table_1"},
    {"id": "table_1", "type": "table"},
    {"id": "backpack_1", "type": "backpack",
     "location": "chair_2"},
    {"id": "chair_2", "type": "chair"}
  ],
  "relations": [
    {"subject": "person_1", "predicate": "on",
     "object": "chair_1"},
    {"subject": "laptop_1", "predicate": "on",
     "object": "table_1"},
    {"subject": "backpack_1", "predicate": "on",
     "object": "chair_2"}
  ]
}
```

The `objects` array lists every entity in the scene with its type, state, and location. The `relations` array lists spatial and functional relationships between entities. This structure is the foundation for all QA generation and claim checking.

Hooks for future evaluation tracks. Even though you only score grounding correctness this summer, each scenario's ground truth should also carry privacy and utility metadata so that future evaluation tracks can use the same scenarios without reformatting them.

```
{
```

```
  "privacy_policy": {
    "allowed_api": "API_1",
    "max_retention_seconds": 30,
    "allowed_output_scope": "localized",
    "backend_export_allowed": false
  },
  "utility_target": {
    "max_latency_ms": 1000,
    "required_claims": ["person_present",
                        "obstacle_present"],
    "false_alarm_cost": "medium",
    "miss_cost": "high"
  }
}
```

You do not need to build checkers for these fields this summer. But include them in your schema so the format does not need to change later. If the field is absent, the evaluator ignores it.

7.2 Structured Claims

When TeLLMe answers a question, its answer is a natural-language sentence. You cannot check a sentence directly, because the same fact can be expressed in many ways ("there is a person sitting" vs. "someone is seated" vs. "one occupant, sitting position"). The solution is to decompose the sentence into **structured claims**, each with a subject, predicate, and object that can be checked against ground truth using a simple lookup.

This decomposition is the bridge between the fuzzy world of natural language and the precise world of deterministic checking. Oscar's research layer uses an LLM to perform this decomposition (claim extraction). The undergrad-built checkers then operate entirely on the structured claims, never touching the original natural language.

```
{
  "claim_id": "claim_001",
  "scene_id": "smartroom_001",
  "subject": "person_1",
  "predicate": "sitting_on",
  "object": "chair_1",
  "source": "TeLLMe",
  "natural_language": "A person is sitting on the chair."
}
```

A single TeLLMe answer may produce multiple claims. For example, "A person is sitting at the desk using a laptop. A backpack is on the chair." produces three claims: person_1 sitting_on chair_1, laptop_1 on table_1, and backpack_1 on chair_2.

7.3 QA Pairs Generated from Templates

Writing questions by hand is slow and inconsistent. If you have 50 scenarios, each with 6 questions, that is 300 questions to write and maintain. Instead, you build a QA **generator** that reads the ground-truth state and automatically produces questions and expected answers from templates. The template says "for every object of type X, ask: Is there a [X] in the room?" The generator fills

in the blanks from ground truth. This scales. It is consistent. And when you add a new scenario, the generator produces questions for it automatically.

The QA generator produces questions and expected answers from the ground-truth state. Each question has an `answer_type` (boolean, count, categorical) and a `grounding_rule` that ties it to the specific ground-truth field it checks against.

```
[
  {
    "question": "Is there a person in the room?",
    "expected_answer": "yes",
    "answer_type": "boolean",
    "grounding_rule": "exists(type=person)"
  },
  {
    "question": "How many people are in the room?",
    "expected_answer": 1,
    "answer_type": "count",
    "grounding_rule": "count(type=person)"
  },
  {
    "question": "Is the laptop on the table?",
    "expected_answer": "yes",
    "answer_type": "boolean",
    "grounding_rule": "relation(laptop,on,table)"
  },
  {
    "question": "What is the person doing?",
    "expected_answer": "sitting",
    "answer_type": "categorical",
    "grounding_rule": "attribute(person,state)"
  }
]
```

The QA generator should also produce **negative questions**. If the ground truth says the backpack is on the chair, then "Is the backpack on the table?" should have expected answer "no." This gives both true and false test cases.

7.4 Three Result Categories

Most evaluation systems use two categories: correct and incorrect. This is misleading because it forces a verdict on claims the system cannot actually check. If TeLLMe says "the person looks tired," is that correct or incorrect? You have no ground-truth label for tiredness. Calling it "incorrect" penalizes TeLLMe unfairly. Calling it "correct" inflates the score. Neither is right.

GroundingEval uses three categories. This trichotomy is inspired by work in formal logic verification (see the VeriCoT paper in Background Reading), but the idea is simple and does not require any formal logic background to understand.

SUPPORTED. The claim is entailed by ground truth. The checker found a matching object, relation, or attribute. You can point to specific ground-truth facts that make this claim true.

CONTRADICTED. The claim contradicts ground truth. For example, TeLLMe says there are

two people but ground truth has one. You can point to specific ground-truth facts that make this claim false.

NOT_EVALUABLE. The claim cannot be checked. Ground truth is insufficient (no label for this attribute), the claim is subjective ("the person looks tired"), the claim type is not yet supported by your checkers, or the claim is about something outside the scenario's scope.

Do not force every sentence into SUPPORTED or CONTRADICTED. Acknowledging NOT_EVALUABLE is essential for honest evaluation. A system that reports 100% accuracy on 20 out of 50 claims, with 30 marked NOT_EVALUABLE, is more honest and more useful than a system that reports 85% accuracy on all 50 by guessing on the hard ones. The NOT_EVALUABLE count also tells you where your ground truth needs to be richer.

7.5 Claim Types and Deterministic Checkers

Start with these claim types. Each has a deterministic checker function.

Claim Type	Example	Checker
Object presence	"There is a chair."	`exists(type=chair)`
Object count	"There are three people."	`count(type=person)`
Attribute	"The chair is red."	compare attribute field
Spatial relation	"The cup is on the table."	relation exists in ground truth
Activity/state	"Person is sitting."	pose/state label match
Temporal change	"The door opened."	compare t1/t2 states
Safety condition	"Path is blocked."	derived rule

Avoid for now: intent, emotion, causality, complex activity recognition, subjective safety claims, and open-ended explanations. Mark those NOT_EVALUABLE.

8 Scenario Package Structure

A scenario is a **reproducible test case**. It captures everything you need to evaluate one situation: what happened, what the ground truth is, what questions were asked, what TeLLMe answered, and how the evaluation scored it. The directory structure is designed so that any student from any team can open a scenario folder, understand it without asking anyone, and reproduce the evaluation.

Every scenario is a self-contained directory.

```
scenarios/smartroom_001/
  README.md
  scenario.json          # what happened, conditions
  ground_truth.json      # objects, relations, attributes
  qa_pairs.json          # generated questions + expected answers
  tellme_answers.json    # TeLLMe's actual answers
  claims.json            # extracted structured claims
  evaluation_results.json
  selected_frames/
  detector_outputs/
  notes.md
```

A student from TeLLMe or Smart Room should be able to open a scenario directory and understand what happened, what was tested, and what the results were.

8.1 Example. scenario.json

```
{
  "scenario_id": "smartroom_001",
  "description": "Single person enters room, sits at desk,
                  laptop on table, backpack on chair",
  "location": "Winlab Smart Room",
  "camera_id": "smartroom-cam1",
  "start_time": "2026-06-15T10:00:00Z",
  "end_time": "2026-06-15T10:05:00Z",
  "conditions": {
    "lighting": "fluorescent",
    "num_subjects": 1,
    "known_difficulties": ["partial occlusion from desk"]
  },
  "source": "physical"
}
```

The `source` field is either `"physical"` (Smart Room) or `"simulated"` (CARLA). For simulated scenarios, ground truth is exported automatically from the simulator state.

9 Team Roles and Workstreams

The team is structured around a principle: **separate the interface from the implementation, and separate the data from the logic**. Oscar defines the schemas and interfaces (what the data looks like, what the API is). Vikhyat implements the deterministic checking logic. Undergrad 2 builds the data pipeline (scenarios, QA generation, converters). Aryan produces the actual test data (scenario cards, labels, adversarial examples). Each person's work feeds into the next, and Oscar's schemas are the contract that holds everything together.

The reason for this separation is practical. If Oscar changes the claim schema, Vikhyat's checkers need to be updated, but Aryan's scenario cards remain valid as long as the ground-truth format stays the same. If Aryan adds a new scenario, Vikhyat's checkers run on it automatically without any code changes. If Undergrad 2 adds a new QA template, it generates questions for all existing scenarios. The data model is the glue.

9.1 Oscar (Grad Student). Architecture and Research Verification

Oscar is the architect and research lead. He defines the data model, supervises all students, and builds the research verification layer.

Responsibilities.

1. Define the claim schema, ground-truth schema, scenario schema, QA schema, and result schema.
2. Define the verifier API (the interface between claim extraction and deterministic checking).

3. Decide what is deterministic (student-built checkers) and what uses LLM/VLM assistance (Oscar's claim extraction layer).
4. Build one end-to-end reference pipeline (scenario → ground truth → QA → mock TeLLMe answer → claim extraction → scoring → report).
5. Review student PRs and prevent scope creep.

Beyond pattern matching. The student-built checkers work by pattern matching: does this object exist, does this relation exist, does this count match. This handles the common cases well, but it is brittle for complex claims. What if TeLLMe says "The person who entered the room is now sitting at the same desk where the laptop was placed earlier"? This claim involves identity, temporal sequencing, and co-reference. A simple pattern-matching checker cannot handle it.

Oscar's research layer explores how to handle these harder cases. One promising direction is the VeriCoT approach (Feng et al., 2025), a neuro-symbolic algorithm that translates natural-language claims into formal logic and uses a theorem prover (Z3) to check whether the claim follows from the ground truth. The key idea is that instead of writing a custom checker for every possible claim type, you translate both the claim and the ground truth into a common formal representation and let a general-purpose solver determine whether one follows from the other. This would be more powerful than pattern matching and still deterministic.

A concrete version of this pipeline might look like the following.

1. **Claim extraction.** Take TeLLMe's natural-language answer. Use an LLM (GPT-4o or Claude) to extract structured claims (subject-predicate-object).
2. **Formalization.** Translate each structured claim into a logical formula that a solver can reason about.
3. **Ground truth as premises.** Translate the `ground_truth.json` into logical assertions. Each object becomes a fact, each relation becomes a predicate.
4. **Entailment check.** Use a solver like Z3 to check whether each claim follows from the ground-truth assertions. This step is deterministic.
5. **Premise trail.** For each verified or rejected claim, output the specific ground-truth facts that support or contradict it. This provides transparency.

Oscar should also look at AlphaVerus (Aggarwal et al., 2024), particularly its Critique phase, for ideas on detecting claims that "pass" trivially because the ground truth is too sparse to contradict them.

The exact methodology is Oscar's research decision. The student-built checkers are the baseline. Oscar's layer is the ceiling.

Deliverables. Reference pipeline, claim extraction prototype, research verification prototype, verifier API definition, and supervision of all student work.

9.2 Vikhyat (Undergrad 1). Deterministic Python Evaluation Library

Vikhyat owns the scoring code. This is the heart of the project. Everything else (scenarios, QA pairs, claim extraction) is preparation. The checkers are where truth is tested.

Everything Vikhyat builds is deterministic Python. No LLM calls. Given the same input, the checkers always produce the same output. This is not a limitation. This is the point. If your checker gives different answers on different runs, it is not a checker, it is another generator. The whole value of GroundingEval is that the checking is mechanical, inspectable, and reproducible.

Tasks.

1. **Answer normalization.** Map TeLLMe's natural-language answers to canonical forms. Examples:
   - "yes" / "yeah" / "correct" → `yes`
   - "one" / "1" → `1`
   - "sitting" / "seated" → `sitting`

   This is an explicit synonym map coded by hand, not an LLM.
2. **Boolean scoring.** Exact match after normalization.
3. **Count scoring.** Parse integer from answer, compare to expected count.
4. **Categorical scoring.** Exact match of category label after normalization.
5. **Relation scoring.** Check that the claimed relation exists in ground truth.
6. **Temporal-event scoring.** Compare event timestamps within a tolerance window.
7. **Metrics aggregation.** Accuracy, precision/recall by claim type, false positive rate, false negative rate, unsupported claim rate, per-scenario score, per-query score.
8. **Unit tests.** Every checker function must have at least 5 unit tests.
9. **CLI runner.** See the CLI tools section below.
10. **JSON result output.** Every evaluation produces a structured `evaluation_results.json`.

Deliverables. Python package with deterministic validators and metrics, CLI tools, unit tests, and sample evaluation results.

9.3 Undergrad 2 (TBD). Scenario and Ground-Truth Data Pipeline

Undergrad 2 owns input data. The goal is not a huge dataset. The goal is a clean, inspectable benchmark.

Tasks.

1. **Scenario templates.** Define reusable scenario structures. A template specifies what objects to place, what actions to perform, and what ground-truth fields to fill.
2. **Ground-truth JSON generation.** Convert Smart Room observations or simulator state into `ground_truth.json` format.
3. **QA template generation.** Write question templates for each claim type (boolean, count, attribute, relation, temporal, safety). The generator fills in object names and expected answers from ground truth.
4. **Positive and negative QA generation.** For every positive question ("Is the backpack on the chair?" → yes), generate a negative variant ("Is the backpack on the table?" → no).

5. **Benchmark manifest.** A master JSON file listing all scenarios, their claim types, and expected pass rates.
6. **Converter scripts.** Convert CARLA simulator state to `ground_truth.json`. Convert Smart Room labels to `ground_truth.json`.

Deliverables. QA generator, scenario templates, converter scripts, and benchmark manifest.

9.4 Aryan (HS). Scenarios, Labels, Adversarial Cases, and Documentation

Aryan produces the test material that the benchmark needs.

Tasks.

1. **Scenario cards.** Write scenario descriptions (what to set up, what happens, expected facts). Example:

```
Scenario: Backpack on chair

Setup:
- Place one chair near the table.
- Put a black backpack on the chair.
- No person in the room. Door is closed.

Expected facts:
- chair present: yes
- backpack present: yes
- backpack on chair: yes
- person present: no
- door open: no
```

2. **Manual labels.** Write `ground_truth.json` for each physical scenario by watching video and recording objects, relations, and events with timestamps. Mark ambiguous cases explicitly, never silently guess.
3. **Expected answers.** Write expected answers for every generated QA pair. Include evidence pointers.
4. **Adversarial examples.** Create scenarios where TeLLMe might fail.

```
TeLLMe says 2 people, ground truth has 1.
TeLLMe says person is sitting, person is standing.
TeLLMe says object is on table, it is next to table.
TeLLMe says room is empty, person is partially occluded.
TeLLMe says door opened, door was already open.
```

5. **QA pass.** Review scenario packages for missing files, inconsistent labels, and prompts without expected answers.
6. **Documentation.** Write README files, scenario naming conventions, and demo scripts.

Deliverables. Scenario cards, labeled ground truth files, adversarial test cases, QA checklists, and documentation.

10 Two Evaluation Modes

You need two modes because they answer different questions and serve different audiences.

10.1 Benchmark Mode

Benchmark mode answers the question: "*Is the system getting better or worse?*" It uses prebuilt scenarios with frozen ground truth, fixed QA pairs, expected answers, and deterministic scoring. Every time you run it, you get the same score for the same system, which means you can track progress over time. If someone changes TeLLMe's prompt template and the benchmark score drops from 87% to 72%, you know the change made things worse. This is the regression test.

Benchmark mode is what you build first because it does not depend on any other team. You can create scenarios, write ground truth, generate QA pairs, and score mock answers entirely within your own team. By the time TeLLMe produces real answers, your benchmark is already working.

Output includes overall accuracy, accuracy by question type, accuracy by scenario type, false positives, false negatives, unsupported answers, ambiguous answers, and per-scenario scores.

10.2 Live / In-the-Wild Mode

Live mode answers the question: "*Does the system work right now, on this scene, in real time?*" A scenario is generated (simulator) or acted out (Smart Room), the system captures observations, TeLLMe answers questions, and GroundingEval scores on the fly. This is the demo.

For simulator scenarios, ground truth is exported automatically from CARLA's Python API (actor positions, types, states). This is the easiest path to live mode because the simulator always knows what is true.

For physical Smart Room scenarios, a human enters or confirms ground truth using a scenario card script. You set up the room ("put a backpack on the chair, have one person sit at the desk"), you tell the system what is true, and TeLLMe answers questions about it. The live checker says "Given the scenario we scripted, did TeLLMe's answer match the expected ground-truth facts?" It does not claim to know reality without ground truth. It only claims to check TeLLMe's answer against the ground truth you provided.

Live mode is harder to build and depends on coordination with other teams. Build it after benchmark mode works.

11 Scaling Ground Truth with Vision-Language Models

The bottleneck in any evaluation system is ground truth. Writing `ground_truth.json` by hand—watching video, identifying every object, annotating every relation—is slow, expensive, and does not scale. If you want 50 scenarios today and 500 next quarter, manual labeling becomes the constraint, not the evaluation logic. The checkers are fast. The data is slow.

The solution is to use a **Vision-Language Model (VLM)** to generate structured scene descriptions, and have the student-built deterministic checkers verify them. This preserves the core principle—*deterministic checkers verify non-deterministic generators*—while removing the manual labeling

bottleneck.

11.1 The Paradigm

A VLM (GPT-4o, Claude with vision, Gemini, or an open-source model like LLaVA) can observe raw sensor data—images, video frames, depth maps—and produce a structured JSON description of the scene. Instead of a human writing "one person, sitting, chair_1, laptop on table_1," the VLM writes it. The VLM is the **description generator**. The student-built checkers are the **description verifiers**.

```
Current pipeline (manual ground truth):
  Human watches video
    -> writes ground_truth.json by hand
    -> QA generator produces questions
    -> deterministic checkers score TeLLMe

Scaled pipeline (VLM-generated ground truth):
  VLM observes raw sensor data
    -> generates ground_truth.json automatically
    -> QA generator produces questions
    -> deterministic checkers score TeLLMe
```

The evaluation infrastructure does not change. The checkers do not care whether a human or a VLM wrote the ground truth. They check the same structured JSON either way. What changes is *how fast you can produce test scenarios.*

11.2 How Student Roles Shift

Role	Current (Manual)	Scaled (VLM)
Ground truth	Aryan watches video, writes `ground_truth.json` by hand	VLM generates structured scene descriptions from raw sensor data
Claim extraction	Oscar's LLM layer extracts claims from TeLLMe's NL output	VLM can generate structured claims directly, bypassing NL
Verification	Vikhyat's deterministic Python checkers	Students still build deterministic comparison logic
Quality gate	Human reviews scenario cards	Students curate a "golden dataset" of human-verified examples to evaluate VLM accuracy

The key insight is that even with VLM-generated descriptions, the student-built deterministic infrastructure remains the quality gate. The VLM is not trusted blindly. Its output is checked, measured, and audited using the same deterministic logic the students are building now.

11.3 Three Uses of VLMs in GroundingEval

1. Automated ground-truth generation. Given camera frames or video from the Smart Room, a VLM generates `ground_truth.json` in the same schema the students already defined. This replaces or augments manual labeling. A human reviews a sample of VLM-generated ground truth for quality, but does not write every file by hand.

2. Direct structured claim generation. Instead of the two-step path (TeLLMe NL answer → Oscar's LLM extracts claims), a VLM can observe the scene and produce structured claims directly in subject-predicate-object format. This bypasses natural-language ambiguity entirely and goes straight to the structured format that deterministic checkers consume.

3. Adversarial scenario generation. A VLM can generate challenging scenarios by identifying edge cases: "What if the person is partially occluded?" "What if two objects overlap?" This scales the adversarial testing that Aryan currently does by hand.

11.4 The Golden Dataset

If a VLM generates ground truth, who checks the VLM? You need a small, highly curated set of human-verified ground truth—a **golden dataset**—to measure how accurate the VLM's descriptions are. This is the meta-evaluation: you are evaluating the evaluator's data source.

Aryan's role shifts from writing every `ground_truth.json` by hand to curating a golden dataset of 20–50 scenarios where every object, relation, and attribute is human-verified to high confidence. You then run the VLM on the same scenarios and measure agreement. If the VLM achieves 95%+ agreement with human labels on the golden dataset, you can trust it to generate ground truth for the remaining 450 scenarios that no human has time to label.

```
Golden dataset workflow:
  1. Aryan curates 20-50 human-verified scenarios
  2. VLM generates ground_truth.json for same scenarios
  3. Vikhyat's checkers compare VLM output to human output
  4. Measure agreement rate (object presence, relations,
     attributes, counts)
  5. If agreement >= 95%, deploy VLM for bulk generation
  6. Periodically re-validate on expanded golden set
```

This is the same "deterministic checkers verify non-deterministic generators" principle applied one level up. The VLM is a non-deterministic generator of ground truth. The golden dataset and deterministic comparison logic verify it.

11.5 Impact on the 12-Week Timeline

VLM-based ground truth generation is not a prerequisite for the summer project. The manual pipeline works and is sufficient for 50 scenarios. But the VLM path should be explored in Phase 5 (Weeks 9–11) as a scaling experiment:

1. Oscar prompts a VLM with Smart Room camera frames and asks it to produce `ground_truth.json` in the defined schema.
2. Aryan compares VLM output to his human-written ground truth for the same scenarios.
3. Vikhyat runs the existing checkers on VLM-generated ground truth to measure agreement with human labels.
4. The team documents the agreement rate, failure modes, and recommendations for scaling.

If the experiment succeeds, next semester's team inherits a pipeline that can generate hundreds of evaluation scenarios with minimal human effort. The deterministic evaluation infrastructure the students build this summer is what makes that scaling possible.

12 CLI Tools

```
# Score TeLLMe answers against ground truth
grounding-eval score \
  --qa qa_pairs.json \
  --answers tellme_answers.json \
  --ground-truth ground_truth.json \
  --out results.json

# Generate QA pairs from ground truth
grounding-eval generate-qa \
  --ground-truth ground_truth.json \
  --out qa_pairs.json

# Generate a scenario from a template
grounding-eval generate-scenario \
  --template person_sitting_at_table \
  --out scenarios/smartroom_001/

# Compute aggregate metrics from results
grounding-eval metrics --results results.json

# Generate a human-readable report
grounding-eval report --results results.json
```

13 Concrete Demo

To make all of this tangible, here is what a working GroundingEval demo looks like end-to-end. Read these examples carefully. They show exactly what the system does, what good output looks like, and what failure looks like. When you are unsure what to build next, come back to these examples and ask: "Does my code produce this output?"

Success case.

```
Scenario: person sitting at desk, laptop on table,
          backpack on chair.

Query: What is happening in the room?

TeLLMe answer:
"A person is sitting at the desk using a laptop.
 A backpack is on the chair."

Extracted claims:
  1. person_1 sitting_at desk_1
  2. laptop_1 on table_1
  3. backpack_1 on chair_1
```

```
Grounding Eval:
  claim 1: SUPPORTED
  claim 2: SUPPORTED
  claim 3: SUPPORTED

Overall grounding score: 3/3
```

Failure case.

```
TeLLMe answer:
"Two people are standing near the door."

Extracted claims:
  1. count(person) = 2
  2. person_1 state = standing
  3. person_1 near door

Grounding Eval:
  claim 1: CONTRADICTED (ground truth: 1 person)
    error type: wrong count
  claim 2: CONTRADICTED (ground truth: sitting)
    error type: wrong state
  claim 3: CONTRADICTED (ground truth: near desk)
    error type: wrong relation

Overall grounding score: 0/3
```

Full multi-dimensional evaluation (future vision).

This is what a complete evaluation looks like when all tracks are implemented. You build the grounding track this summer. The utility and privacy tracks show where the infrastructure is heading.

```
Scenario: person enters room, sits at table,
          opens laptop.
Question: What is happening in the room?
TeLLMe:   A person is sitting at the table
          using a laptop.

--- Grounding Eval (your core) ---
  person present:      SUPPORTED
  person sitting:      SUPPORTED
  person near table:   SUPPORTED
  laptop present:      SUPPORTED
  using laptop:        NOT_EVALUABLE
    (no interaction labels in ground truth)

--- Utility Eval (future track) ---
  answer useful for room-status app:  yes
  answer useful for emergency app:    incomplete
  latency:          1.2s
  coverage:         4/5 required facts
```

```
--- Privacy Eval (future track) ---
  used API 1:              yes
  raw frames exported:     no
  output localized only:   yes
  retention <= maxEC:      yes
  backend call attempted: no
```

The grounding track is the core. The other tracks are additional metric modules that run on the same scenario data. That is why the scenario format includes hooks for privacy policies and utility targets even though you are not scoring them yet.

14 12-Week Schedule

14.1 Phase 1. Define the Interface (Weeks 1–2)

Oscar defines the claim schema, ground-truth schema, scenario schema, QA schema, and result schema. Oscar reads the VeriCoT paper (Feng et al., 2025) and explores whether its approach to formal verification could work for scene claims.

Students clone CityOS, study `eye-in-the-sky` output format, and set up the `groundingeval` repo with directories `schemas/`, `scenarios/`, `scripts/`, `examples/`, and `reports/`.

Aryan writes the first scenario card ("person sitting at desk"). Vikhyat writes the first JSON schema (`ground_truth.schema.json`) and a validation script.

Milestone. All schemas defined. First scenario card written. Repo structure in place.

14.2 Phase 2. Build Deterministic Checkers (Weeks 3–5)

Vikhyat implements the six core checker functions.

```
def check_presence(ground_truth, object_type):
    return any(obj["type"] == object_type
               for obj in ground_truth["objects"])

def check_count(ground_truth, object_type, expected):
    actual = sum(1 for obj in ground_truth["objects"]
                 if obj["type"] == object_type)
    return actual == expected

def check_relation(ground_truth, subject_type,
                   predicate, object_type):
    for rel in ground_truth["relations"]:
        s = next((o for o in ground_truth["objects"]
                  if o["id"] == rel["subject"]), None)
        o = next((o for o in ground_truth["objects"]
                  if o["id"] == rel["object"]), None)
        if (s and o and s["type"] == subject_type
            and rel["predicate"] == predicate
            and o["type"] == object_type):
```

```
            return True
    return False

def check_attribute(ground_truth, object_type,
                    attribute, expected_value):
    for obj in ground_truth["objects"]:
        if (obj["type"] == object_type
            and obj.get(attribute) == expected_value):
            return True
    return False
```

Each checker returns SUPPORTED, CONTRADICTED, or NOT_EVALUABLE. Vikhyat writes unit tests for every checker. Vikhyat builds the CLI runner (`grounding-eval score`).

Aryan writes scenario cards and labels for 3 scenarios. Undergrad 2 begins the QA template generator.

Milestone. Checkers pass all unit tests. CLI runner works on one scenario end-to-end.

14.3 Phase 3. Build Benchmark Scenarios (Weeks 5–7)

Undergrad 2 builds the QA generator (`grounding-eval generate-qa`). The generator reads `ground_truth.json` and produces positive and negative QA pairs from templates.

Aryan creates 10+ scenarios covering five families.

A. Basic occupancy. Empty room, 1 person, 2 people, enter, exit, sit.

B. Event detection. Enter/exit doorway, brief appearance, long stillness.

C. Counting. Max occupancy, entry/exit counts, time-to-empty.

D. Relation and attribute. Object on table, person at desk, door open/closed.

E. Failure and ambiguity. Occlusion, poor lighting, partial body, close groups.

Vikhyat adds metrics aggregation (accuracy by claim type, per-scenario scores). Aryan creates adversarial examples for each scenario family.

Milestone. 10+ scenarios with ground truth, QA pairs, and expected answers. QA generator produces valid output. Metrics aggregation works.

14.4 Phase 4. Integrate TeLLMe and Claim Extraction (Weeks 7–9)

Oscar defines the TeLLMe output format and claim extraction path. Start with **mock TeLLMe outputs** (hand-written answers for each QA pair). Do not wait for the real TeLLMe app to be stable.

Oscar builds the claim extraction prototype. Given a TeLLMe NL answer, the extractor uses GPT-4o to produce structured claims (subject-predicate-object JSON). Oscar begins exploring formal verification approaches for complex claims.

Vikhyat wires benchmark mode end-to-end. The CLI runs on all scenarios, produces `evaluation_results.json` for each, and generates an aggregate report.

Milestone. First benchmark report on 10+ scenarios with mock TeLLMe answers. Claim extraction prototype produces structured claims from NL answers.

14.5 Phase 5. Live Mode and Expansion (Weeks 9–11)

Build the live evaluation path for either the Smart Room or CARLA simulator (or both).

For CARLA, Undergrad 2 writes a converter that exports CARLA actor state to `ground_truth.json`. For Smart Room, Aryan writes scenario cards and ground truth from physical observations.

Expand the benchmark to 50 scenarios and 300+ QA pairs. Oscar connects claim extraction to his research verification prototype.

Connect to real Smart Room detector output (coordinate with Crystal and Alan). Connect to real TeLLMe output (coordinate with Sravya).

Milestone. Live demo path working. Benchmark expanded. Real detector and/or TeLLMe outputs evaluated.

14.6 Phase 6. Demo and Handoff (Week 12)

Write handoff documentation for the Smart Room team ("how to export ground truth") and the TeLLMe team ("how to export answers," "how to add a new scenario," "how to run the evaluator"). A new student should be able to run the evaluator without asking Vikhyat.

Polish all scripts. Run the end-to-end demo. Present results.

Milestone. Working demo, final benchmark report, handoff documentation.

15 Joint Deliverables

J1. Minimum viable benchmark. 10 clean validated scenarios. Every scenario has ground truth, QA pairs, expected answers, and evaluation results. One aggregate report.

J2. Smart Room integration package. Ground-truth format spec, example, converter instructions, and evaluation commands. The Smart Room team can produce a `ground_truth.json` for any scenario.

J3. TeLLMe integration package. Answer format, claim extraction spec, evaluation commands, and evidence-link format. TeLLMe can run a batch of prompts and get scored.

J4. Final evaluation suite. 50 scenarios, 300+ QA pairs, claim-level evaluation results, error taxonomy, adversarial examples, demo-ready scenarios, and handoff documentation.

16 Coordination with Other Teams

Team	Interface
Smart Room (2)	Provides raw sensor data, detector outputs, and scene metadata. GroundingEval provides the ground-truth format spec and claim-level evaluation results.
TeLLMe (4)	Provides canonical QA pairs and the evaluator. TeLLMe exports answers in a format that claim extraction can process.
CARLA (5)	Provides simulated scenarios with known ground truth (actor positions, types, states). GroundingEval provides the ground-truth format spec and automated QA generation.
TraceFix (1B+1C)	No direct interface. TraceFix verifies coordination protocols. GroundingEval verifies scene-level claims.

17 Related Work and Prior Art

Pieces of what GroundingEval does already exist in the research literature, but no single benchmark matches the full picture. Understanding the prior art helps you see what to borrow, what is already solved, and where the novelty lies.

17.1 CLEVR. Deterministic QA from Synthetic Scenes

CLEVR (Stanford) is probably the closest conceptual ancestor for the deterministic QA-generation part. It uses synthetic rendered scenes, ground-truth scene JSON, generated questions, functional programs, and deterministic answers. The official dataset generator takes ground-truth scene information and produces questions, answers, and programs that compute the answers. That is basically the pattern you want for the QA generator.

What to borrow. The pattern: scene ground truth → question templates → functional programs → deterministic answers. Study how CLEVR generates compositional questions from structured scene representations.

What is missing. CLEVR is not live, not smart-room-specific, not multimodal sensor/runtime oriented, and not privacy/utility/policy aware. It is a static offline dataset, not infrastructure for evaluating a deployed application.

https://cs.stanford.edu/people/jcjohns/clevr/

17.2 GQA. Scene-Graph QA on Real Images

GQA (Stanford) is the closest real-image version. It uses real-world images, scene graphs, structured question semantics, and functional programs. The dataset has millions of compositional questions derived from scene graphs, and it includes metrics for consistency, grounding, and plausibility.

What to borrow. GQA is a strong precedent for scene-graph-grounded compositional QA. Its

metrics (consistency, grounding, plausibility) are worth studying. You are doing something similar, but for smart-room and simulator scenarios rather than static images.

What is missing. Static images only. Not live, not tied to application answers from TeLLMe, and not privacy/API compliance aware.

https://cs.stanford.edu/people/dorarad/gqa/about.html

17.3 EmbodiedQA. QA Grounded in Simulated 3D Environments

EmbodiedQA (Georgia Tech / Facebook AI) introduced visual questions and answers grounded in a simulated 3D environment (House3D). Questions can be represented as functional programs that are programmatically generated and executed against the environment to determine answers. This is probably the closest "simulator + QA + ground truth" benchmark.

What to borrow. The pattern of grounding questions in a 3D environment where ground truth comes from the simulator state. This is what your CARLA integration does.

What is missing. Mainly about embodied navigation, not a general smart-room evaluation harness. Not privacy/utility/policy focused and not designed as live benchmark infrastructure for a deployed application.

https://embodiedqa.org/

17.4 AI2-THOR. Interactive Indoor Simulation

AI2-THOR (Allen Institute) provides near-photorealistic indoor scenes where agents can navigate and interact with objects. It has been used for interactive QA and embodied action understanding benchmarks. Configurable object states and interactive scenarios make it a natural fit for smart-room evaluation.

What to borrow. Indoor simulated environments with configurable object states and ground truth from simulator state. For the smart-room part, AI2-THOR may actually be a better simulator than CARLA, while CARLA is better for streetscape/traffic scenarios.

What is missing. Not a TeLLMe-style QA evaluator. Not tied to runtime privacy constraints.

https://ai2thor.allenai.org/

17.5 AGQA. Compositional Spatio-Temporal QA over Video

AGQA generates question-answer pairs from action/event structure and evaluates reasoning over temporal actions, spatial relations, and objects. This is useful if TeLLMe answers questions like "what changed?", "did someone enter?", or "did the person pick up the object?"

What to borrow. Temporal event QA, action reasoning, and compositional spatio-temporal questions. These map to your temporal-change and event-detection claim types.

What is missing. Offline video benchmark. Not live, not simulator-generated, and not system-evaluation infrastructure.

https://arxiv.org/abs/2103.16002

17.6 MLLM Hallucination Benchmarks

Recent benchmarks for multimodal LLM hallucination (POPE, MME, MMHal-Bench, Hallusion-Bench, and others) test whether a model invents objects, attributes, relations, or answers unsupported by an image. They are useful for test categories: object existence, object count, attribute correctness, relation correctness, and refusal / none-of-the-above.

What to borrow. Error categories for hallucination, especially the distinction between object hallucination, attribute hallucination, and relation hallucination. These map directly to your error types (hallucination, wrong count, wrong relation, wrong state).

What is missing. Mostly static image/video benchmarks. Not live smart-room/simulator infrastructure. Not privacy/utility/API compliance aware.

https://arxiv.org/abs/2404.18930 (survey)

17.7 CityOS Evaluation. Privacy/Utility Curves

The CityOS paper is not a grounding benchmark, but it gives the right model for the privacy/utility side. It evaluates API mechanisms using privacy/utility curves: API 1 sniffing exposure versus `maxEC`, API 2 DP counting accuracy versus aggregation window and sensitivity, and API 3 OD accuracy versus privacy budget and self-identification quality. These are exactly the kinds of evaluations that PrivacyEval and UtilityEval should eventually produce.

17.8 Where GroundingEval Fits

The intellectually honest positioning is:

> *"Existing benchmarks evaluate visual reasoning, embodied QA, hallucination, or DP utility separately. GroundingEval combines these ideas into a live smart-room/simulator evaluation harness for TeLLMe-style multimodal applications, with deterministic scoring and hooks for privacy, utility, and policy compliance."*

The novelty is not "we generate QA from ground truth." CLEVR, GQA, EQA, and AGQA already do versions of that. The novelty is:

1. A live smart-room + simulator evaluation harness (not an offline dataset).
2. Deterministic claim-level grounding evaluation for deployed applications (not just model benchmarking).
3. Unified benchmark and in-the-wild evaluation modes.
4. Integration with privacy/utility/policy constraints from CityOS.
5. Testing not only whether the model answered correctly, but whether the *application* stayed within its allowed data-access contract.

Most VQA benchmarks ask: "Did the model answer correctly?" GroundingEval can ask: "Did the system answer correctly, usefully, quickly, and without violating the privacy contract?" That is the stronger angle.

18 Background Reading

18.1 For Implementation Inspiration

1. **CLEVR dataset generator** at https://cs.stanford.edu/people/jcjohns/clevr/. Study how it generates deterministic QA from scene JSON. This is the model for your QA generator.
2. **GQA scene graphs and functional programs** at https://cs.stanford.edu/people/dorarad/gqa/about.html. Study the grounding and consistency metrics.
3. **EmbodiedQA / EQA** at https://embodiedqa.org/. Study how QA is grounded in a simulated 3D environment.
4. **AGQA** at https://arxiv.org/abs/2103.16002. Study temporal event QA if you are doing questions like "what changed" or "did someone enter."
5. **POPE / hallucination benchmarks**. Study the survey at https://arxiv.org/abs/2404.18930 for error categories and negative example design.
6. **CityOS evaluation section** (pages 11–13 of the CityOS paper). Study the privacy/utility curves for the UtilityEval and PrivacyEval tracks.

18.2 For Research Directions

1. **VeriCoT** (Feng et al., 2025), "VeriCoT: Neuro-Symbolic Chain-of-Thought Validation via Logical Consistency Checks." A promising approach to formal claim verification that Oscar may draw from. All students should read the introduction to understand the three result categories (entailed, contradicted, ungrounded), which inspired GroundingEval's SUPPORTED / CONTRADICTED / NOT_EVALUABLE.
2. **AlphaVerus** (Aggarwal et al., 2024), "AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement." Oscar should look at the Critique phase (Section 3.1) for ideas on detecting claims that pass trivially because ground truth is too sparse.
3. **Z3 SMT solver** documentation at https://github.com/Z3Prover/z3. Oscar may use Z3 for formal claim checking. The `z3-solver` Python package provides the API.

18.3 For Everyone

1. **CityOS repo**, `apps/eye-in-the-sky/` for understanding detector output format.
2. **JSON Schema specification** at https://json-schema.org/. All students should understand how to write and validate JSON schemas.

19 What to Avoid

These are common failure modes for evaluation projects. Each one sounds reasonable in the moment and leads to wasted weeks.

Do not turn this into a generic labeling tool. Labeling is a means, not the end. The goal is a claim-level evaluation harness, not a labeling platform.

Do not pursue open-ended model training. You are not training any model. You are building evaluation infrastructure that measures how well an existing model (TeLLMe) performs.

Do not make the undergrads build the research verification system. Claim extraction and formal verification are Oscar's research layer. Vikhyat and Aryan build deterministic checkers and scenarios. Do not blur this boundary.

Do not start with arbitrary natural-language answers. Start with controlled QA templates where the expected answer is unambiguous ("yes," "1," "sitting"). Free-form answers like "The room appears to have moderate occupancy with some activity" are much harder to evaluate and should come later.

Do not start with full CARLA complexity. Start with a single room, a few objects, and simple questions. Add complexity only after the basic pipeline works end-to-end.

Do not evaluate correctness by asking another LLM. This undermines the entire design principle. If you need an LLM to check TeLLMe's answer, you have not built an evaluation system, you have built another generator. The only exception is Oscar's claim *extraction* step, where an LLM converts natural language into structured claims. But the *checking* of those claims against ground truth must be deterministic.

Do not evaluate only happy-path scenarios. Adversarial and failure cases are where evaluation is most valuable. If TeLLMe gets easy questions right, that is expected. If it gets hard questions wrong, that tells you where to improve.

Do not require deep changes to CityOS, ConcordFS, or TraceFix. You are building evaluation infrastructure alongside the stack, not modifying it.